

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 18:03 UTC

Overall Assessment

The code review has identified 12 findings with a risk score of 45, indicating a high level of risk. The majority of the findings are critical, related to security vulnerabilities such as SQL injection, arbitrary code execution, and hardcoded sensitive data.

Merge readiness: Not Ready

Code health: 0/100

Findings Summary

Total findings: 12

Risk score: 45

Severity breakdown: Low 0, Medium 0, High 3, Critical 9

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
Security	Critical	1	Hardcoded password	Use environment variables or a secure secrets management system to store sensitive data
Security	Critical	3	Hardcoded API key	Use environment variables or a secure secrets management system to store sensitive data
Security	Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection attacks
Security	Critical	7	Arbitrary code execution via eval	Avoid using eval and instead use safer alternatives such as ast.literal_eval or json.loads

Type	Severity	Line	Message	Recommendation
Security	Critical	9	Arbitrary code execution via exec	Avoid using exec and instead use safer alternatives such as subprocess.run with a whitelist of allowed commands
Security	High	11	Subprocess execution with shell=True	Use subprocess.run with shell=False and pass the command as a list of arguments to prevent shell injection attacks

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
SQL Injection	Critical	High	Use parameterized queries to prevent SQL injection attacks
Arbitrary Code Execution	Critical	High	Avoid using eval and exec, and instead use safer alternatives such as ast.literal_eval or json.loads
Hardcoded Sensitive Data	Critical	High	Use environment variables or a secure secrets management system to store sensitive data
Command Injection	Critical	High	Use subprocess.run with shell=False and pass the command as a list of arguments to prevent shell injection attacks
Hardcoded Password	High	Medium	Store passwords in environment variables
Hardcoded API Key	High	Medium	Store API keys in environment variables
Subprocess Execution with shell=True	High	Medium	Use subprocess.run with shell=False and pass the command as a list of arguments to prevent shell injection attacks

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess
```

```
subprocess.run("dir", shell=True)
```